PROGRAMMING LANGUAGES

Python

The AI engineer's core language, from object model and comprehensions to async, typing, and interview-ready idioms.

python · programming-languages · oop · async · typing · interview-prep · data-engineering

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What Python Is & Why It Matters
2. Setup & Environment Management
3. Core Data Types & The Object Model
4. Control Flow & Pattern Matching
5. Comprehensions & Iteration Idioms
6. Functions, Arguments & Closures
7. Object-Oriented Programming
8. Decorators & Context Managers
9. Generators, Iterators & Lazy Data
10. Type Hints & Modern Typing
11. Errors, Exceptions & Robustness
12. Concurrency: Threads, asyncio & Processes
13. Standard Library Essentials
14. Testing with pytest
15. Cheat Sheet & Comparison Reference
16. Best Practices & Common Pitfalls
17. Interview Questions & Answers
18. Further Learning

What Python Is & Why It Matters

Python is a high-level, dynamically typed, garbage-collected language whose defining promise is that code should be as readable as pseudocode. For a data/AI engineer it is the lingua franca: the same language drives quick data exploration in a notebook, production ETL, model training, and the web service that serves the model.

Python trades raw execution speed for programmer speed and a colossal ecosystem. The heavy numerical work (NumPy, PyTorch, pandas) runs in C/C++/CUDA under the hood; Python is the *glue* that orchestrates it. Understanding where the boundary sits — pure-Python loops are slow, vectorized library calls are fast — is the single most important mental model for performance.

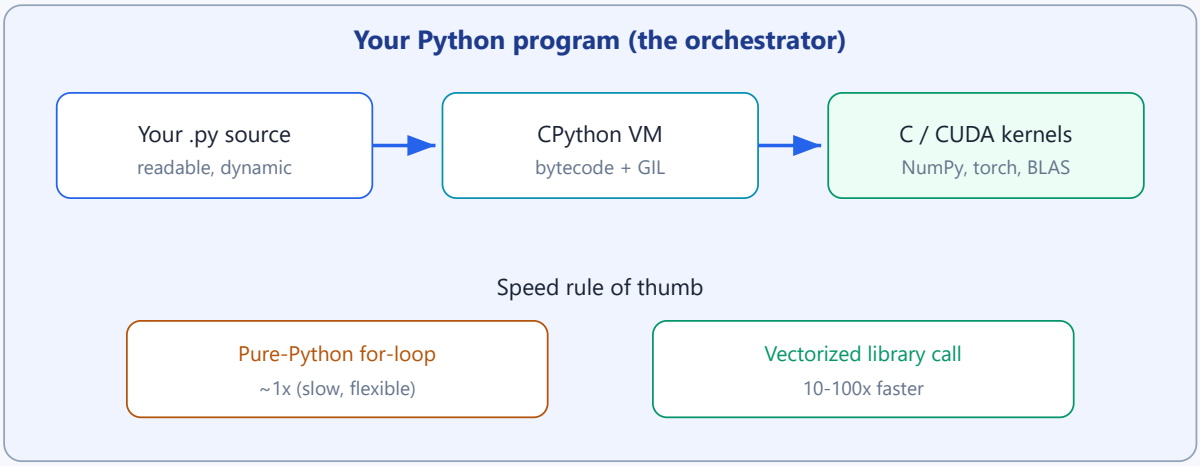


Fig 1 — Python is the conductor; the compute-heavy sections delegate to compiled C/CUDA.

> **Version**
> Target **Python 3.12+** for new work in 2026 (3.13 adds an experimental free-threaded, no-GIL build). Python 2 is long dead. Prefer features available in 3.10+ (structural pattern matching, better error messages, `X | Y` type unions).

Setup & Environment Management

The cardinal rule: **never install packages into the system interpreter**. Each project gets an isolated virtual environment so dependency versions never collide.

BASH

```
# Check your interpreter
python --version            # or python3 on macOS/Linux

# Create and activate a virtual environment (stdlib venv)
python -m venv .venv
source .venv/bin/activate   # Windows: .venv\Scripts\activate

# Install packages into the active venv
pip install numpy pandas requests
pip freeze > requirements.txt      # snapshot exact versions
pip install -r requirements.txt    # reproduce elsewhere

# Modern fast alternative: uv (Rust-based, ~10-100x faster than pip)
uv venv && uv pip install pandas
uv run script.py            # auto-manages the environment
```

VENV + PIP

Built in, universal, zero extra install. The safe default.

UV / POETRY

Lockfiles, dependency resolution, project metadata in `pyproject.toml`. Best for apps.

CONDA

Manages non-Python binaries (CUDA, MKL) too. Common in data science / Anaconda.

> ✦ **Tip**
> Add `.venv/` to `.gitignore`. Commit `requirements.txt` (or `pyproject.toml` + lockfile), never the environment folder itself.

Core Data Types & The Object Model

Everything in Python is an object with an identity, a type, and a value. Variables are *names bound to objects*, not boxes holding values — this explains most aliasing surprises.

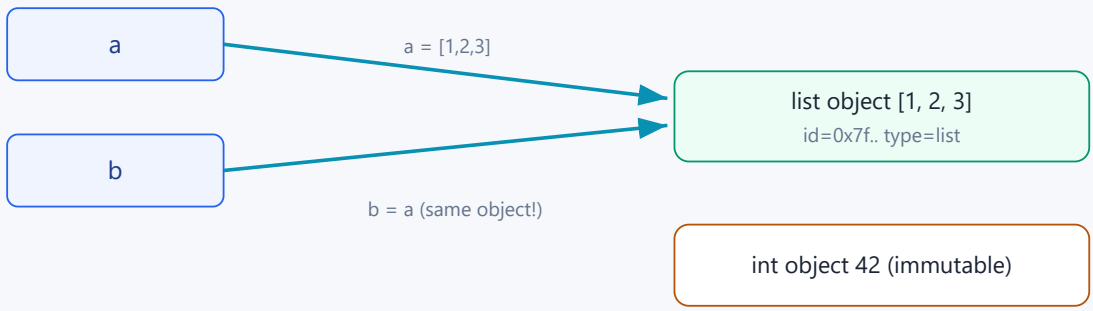


Fig 2 — Names reference objects. `b = a` makes both names point at one list; mutating through `b` is visible via `a`.

PYTHON

```python
# Numbers: int has arbitrary precision, no overflow
2 ** 100                    # 1267650600228229401496703205376

# Strings are immutable Unicode; f-strings are the standard for formatting
name, score = "Ada", 98.5
print(f"{name} scored {score:.1f}%")   # Ada scored 98.5%

# The core containers
nums   = [1, 2, 3]             # list  - ordered, mutable
point  = (10, 20)              # tuple - ordered, immutable (hashable)
seen   = {1, 2, 3}             # set   - unordered, unique, fast membership
lookup = {"a": 1, "b": 2}      # dict  - key -> value, insertion-ordered (3.7+)

# Mutable vs immutable: the #1 aliasing gotcha
a = [1, 2, 3]
b = a                # b is the SAME list
b.append(4)
print(a)             # [1, 2, 3, 4]  -- surprised? see Fig 2
c = a.copy()         # shallow copy -> independent top level
```

Type	Mutable?	Ordered?	Typical use	Lookup cost
`list`	Yes	Yes	Sequence you grow/edit	O(n) search, O(1) index
`tuple`	No	Yes	Fixed record, dict key	O(n)
`dict`	Yes	Insertion	Keyed lookup	O(1) average
`set`	Yes	No	Membership, dedup	O(1) average
`str`	No	Yes	Text	O(n)

Control Flow & Pattern Matching

PYTHON

```python
# Truthiness: empty containers, 0, None, "" are falsy
items = []
if not items:
    print("empty")

# Ternary and chained comparisons
grade = "pass" if score ≥ 50 else "fail"
if 0 ≤ score ≤ 100:           # chained, reads like math
    pass

# for-else: else runs only if the loop did NOT break
for x in nums:
    if x == 99:
        break
else:
    print("99 not found")

# Structural pattern matching (3.10+)
def handle(event):
    match event:
        case {"type": "click", "x": x, "y": y}:
            return f"click at {x},{y}"
        case {"type": "key", "code": code}:
            return f"key {code}"
        case [first, *rest]:           # sequence + capture
            return f"list starting {first}"
        case _:                         # wildcard / default
            return "unknown"
```

Comprehensions & Iteration Idioms

Comprehensions are the idiomatic, fast way to build collections. Prefer them to manual `append` loops — clearer and often quicker.

PYTHON

```python
squares   = [x*x for x in range(10)]                 # list
evens     = [x for x in range(20) if x % 2 == 0]     # with filter
grid      = [(r, c) for r in range(3) for c in range(3)]  # nested
by_len    = {w: len(w) for w in ["a", "bb", "ccc"]}  # dict comp
unique    = {x % 3 for x in range(10)}               # set comp

# Generator expression: lazy, memory-efficient (parens, not brackets)
total = sum(x*x for x in range(1_000_000))           # no giant list built

# Essential built-ins for iteration
for i, val in enumerate(nums, start=1):    # index + value
    print(i, val)
for a, b in zip([1, 2], ["x", "y"]):       # parallel iteration
    print(a, b)
from itertools import chain, groupby, count
list(chain([1, 2], [3, 4]))                # [1,2,3,4]
```

> ⚠ **Pitfall**
>
> Comprehensions build the whole collection in memory. For a huge or infinite stream use a *generator* expression (round parentheses) so values are produced one at a time.

Functions, Arguments & Closures

PYTHON

```python
def connect(host, port=5432, *args, timeout=30, **kwargs):
    """positional, default, *args (extra positionals),
       keyword-only (after *), **kwargs (extra keywords)."""
    ...

# Force keyword-only / positional-only args
def area(width, height, *, unit="cm"):   # unit MUST be passed by name
    return width * height

def divide(a, b, /):                      # a, b are positional-only
    return a / b

# Unpacking at the call site
opts = {"port": 6543, "timeout": 10}
connect("db.local", **opts)

# Closures capture variables by reference
def make_counter():
    count = 0
    def inc():
        nonlocal count      # rebind the enclosing variable
        count += 1
        return count
    return inc
c = make_counter(); c(); c()   # 1, 2
```

Classic bug

Never use a mutable default argument. `def f(items=[]):` shares ONE list across all calls. Use `def f(items=None): items = items or []` instead.

Object-Oriented Programming

PYTHON

```python
from dataclasses import dataclass, field

@dataclass                        # auto __init__, __repr__, __eq__
class Point:
    x: float
    y: float
    tags: list = field(default_factory=list)   # safe mutable default
    def dist(self) -> float:
        return (self.x**2 + self.y**2) ** 0.5

# Inheritance, super(), and properties
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self) -> str:
        raise NotImplementedError

class Dog(Animal):
    def speak(self):
        return "woof"

class Account:
    def __init__(self, balance):
        self._balance = balance
    @property                     # computed, read like an attribute
    def balance(self):
        return self._balance
    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("negative balance")
        self._balance = value

# Dunder methods make objects behave like built-ins
class Vec:
    def __init__(self, x, y): self.x, self.y = x, y
    def __add__(self, o):     return Vec(self.x+o.x, self.y+o.y)
    def __repr__(self):       return f"Vec({self.x}, {self.y})"
    def __len__(self):        return 2
```

@DATACLASS

Removes boilerplate for data-holding classes. Add `frozen=True` for immutability, `slots=True` for memory savings.

@PROPERTY

Expose computed/validated values with attribute syntax — no `get_x()` / `set_x()` noise.

Decorators & Context Managers

A decorator is a function that takes a function and returns a replacement — the mechanism behind `@property`, `@staticmethod`, Flask routes, and caching.

PYTHON

```python
import functools, time

def timed(func):
    @functools.wraps(func)          # preserve name/docstring
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__}: {time.perf_counter()-t0:.4f}s")
        return result
    return wrapper

@timed
def slow(): time.sleep(0.1)

# functools.cache memoizes automatically
@functools.cache
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

# Context managers guarantee cleanup via with-blocks
with open("data.txt") as f:         # file closed even on exception
    text = f.read()

from contextlib import contextmanager
@contextmanager
def timer(label):
    t0 = time.perf_counter()
    yield                           # body of the with-block runs here
    print(f"{label}: {time.perf_counter()-t0:.3f}s")

with timer("load"):
    data = [x for x in range(10**6)]
```



Generators, Iterators & Lazy Data

Generators produce values on demand with `yield`, holding constant memory regardless of stream size — ideal for pipelines over huge datasets or files.

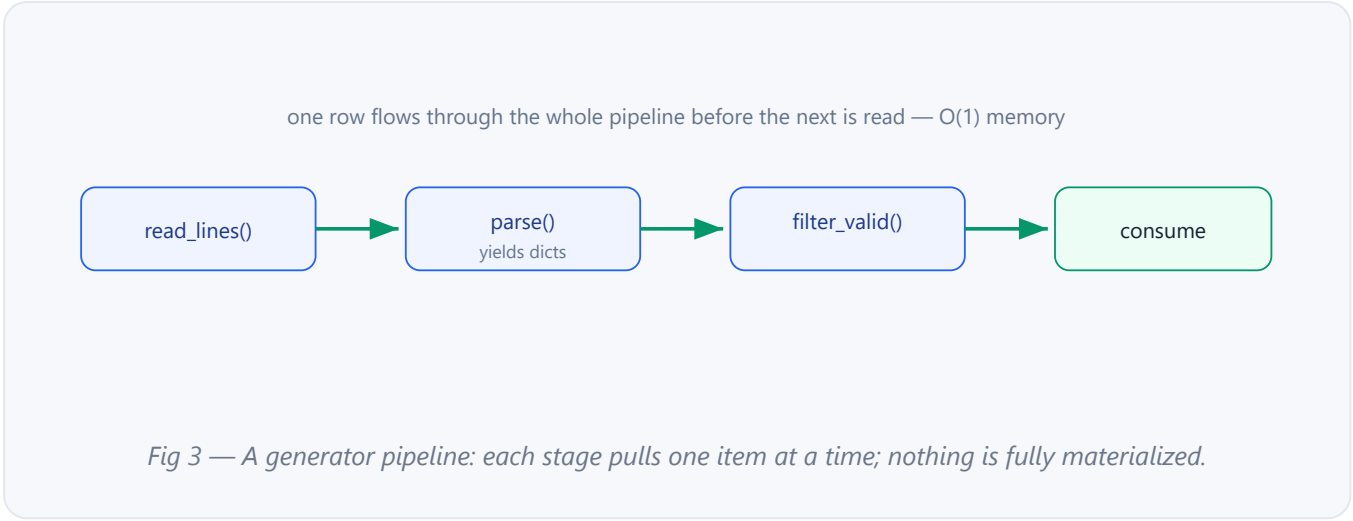


Fig 3 — A generator pipeline: each stage pulls one item at a time; nothing is fully materialized.

PYTHON

```python
def read_lines(path):
    with open(path) as f:
        for line in f:              # file object is itself a lazy iterator
            yield line.rstrip()

def parse(lines):
    for line in lines:
        yield {"raw": line, "len": len(line)}

# Compose lazily — no intermediate lists, works on a 100GB file
pipeline = (r for r in parse(read_lines("big.log")) if r["len"] > 0)
first = next(pipeline)              # pulls exactly one record

# yield can also receive values and produce infinite streams
def counter(start=0):
    n = start
    while True:
        yield n
        n += 1
```

Type Hints & Modern Typing

Hints are optional and not enforced at runtime, but they power editor autocomplete, catch bugs via `mypy` / `pyright`, and document intent. Strongly recommended for any code others will read.

PYTHON

```python
from typing import Optional, Callable, TypeVar, Protocol
from collections.abc import Iterable, Sequence

def mean(xs: Sequence[float]) -> float:
    return sum(xs) / len(xs)

# 3.10+ union syntax and built-in generics
def find(items: list[int], target: int) -> int | None:
    return items.index(target) if target in items else None

# TypeVar for generic functions
T = TypeVar("T")
def first(xs: list[T]) -> T:
    return xs[0]

# Protocol = structural typing ("duck typing" checked statically)
class SupportsClose(Protocol):
    def close(self) -> None: ...

def shutdown(resource: SupportsClose) -> None:
    resource.close()
```

BASH

```bash
pip install mypy
mypy --strict yourmodule.py     # static type check, no execution
```

Errors, Exceptions & Robustness

PYTHON

```python
class ConfigError(Exception):
    """Custom exceptions inherit from Exception."""

def load(path):
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        raise ConfigError(f"missing {path}") from None   # clean chain
    except PermissionError as e:
        raise ConfigError("no access") from e             # preserve cause
    else:
        print("read succeeded")     # runs if no exception
    finally:
        print("always runs")        # cleanup

# EAFP over LBYL: "easier to ask forgiveness than permission"
try:
    value = config["timeout"]       # idiomatic
except KeyError:
    value = 30
# ... instead of:  if "timeout" in config:  ...  (a race in concurrent code)

# Exception groups (3.11+) for concurrent errors
try:
    ...
except* ValueError as eg:           # handle a subset of grouped errors
    ...
```

> ⚠ **Never**
> write a bare `except:` — it swallows `KeyboardInterrupt` and hides bugs. Catch the narrowest exception you can handle, and always log before re-raising.

Concurrency: Threads, asyncio & Processes

The **GIL** (Global Interpreter Lock) lets only one thread execute Python bytecode at a time. So threads help I/O-bound work (waiting on network/disk) but not CPU-bound work — for CPU parallelism use multiple processes.

Workload	Tool	Why
Many network/disk waits	`asyncio` or `ThreadPoolExecutor`	Overlap waiting; GIL released during I/O
Heavy CPU (math, parsing)	`ProcessPoolExecutor` / `multiprocessing`	Separate processes bypass the GIL
Thousands of concurrent sockets	`asyncio`	One thread, cooperative scheduling, low overhead

PYTHON

```python
import asyncio, aiohttp

async def fetch(session, url):
    async with session.get(url) as resp:
        return await resp.text()

async def main(urls):
    async with aiohttp.ClientSession() as session:
        # gather runs all requests concurrently on one thread
        return await asyncio.gather(*(fetch(session, u) for u in urls))

# asyncio.run(main(["https://example.com"] * 50))

# CPU-bound work: use processes
from concurrent.futures import ProcessPoolExecutor
def heavy(n): return sum(i*i for i in range(n))
with ProcessPoolExecutor() as pool:
    results = list(pool.map(heavy, [10**6] * 8))   # true parallelism
```

2026

Python 3.13 ships an experimental free-threaded (no-GIL) build. It is not yet the default; write GIL-aware code and choose the process/async tool that matches your workload.

Standard Library Essentials

COLLECTIONS

`defaultdict`, `Counter`, `deque`, `namedtuple`

ITERTOOLS

lazy combinatorics: `product`, `combinations`, `groupby`, `islice`

FUNCTOOLS

`cache`, `reduce`, `partial`, `wraps`

PATHLIB

object-oriented filesystem paths — prefer over `os.path`

JSON / CSV

serialize and read tabular/document data

DATETIME

timezone-aware times, `timedelta` arithmetic

PYTHON

```python
from collections import Counter, defaultdict, deque
from pathlib import Path
import json

words = "a b a c a b".split()
Counter(words).most_common(1)         # [('a', 3)]

groups = defaultdict(list)
for w in words:
    groups[w[0]].append(w)            # no KeyError on first access

queue = deque([1, 2, 3])
queue.appendleft(0); queue.pop()      # O(1) at both ends

# pathlib reads like English
cfg = Path.home() / ".config" / "app.json"
if cfg.exists():
    data = json.loads(cfg.read_text())
```

Testing with pytest

PYTHON

```python
# file: test_math.py   →   run with:  pytest -q
import pytest
from mymath import divide

def test_divide():
    assert divide(6, 2) == 3

def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        divide(1, 0)

@pytest.mark.parametrize("a,b,expected", [
    (2, 2, 1), (9, 3, 3), (10, 5, 2),
])
def test_cases(a, b, expected):
    assert divide(a, b) == expected

@pytest.fixture
def sample_data():
    return {"users": [1, 2, 3]}       # reusable setup, injected by name

def test_uses_fixture(sample_data):
    assert len(sample_data["users"]) == 3
```

Tip

Run `pytest --cov=mypkg` for coverage. Keep tests fast and deterministic; mock external services with `unittest.mock` or `pytest-mock`.

Cheat Sheet & Comparison Reference

Task	Idiom
Swap variables	`a, b = b, a`
Merge dicts (3.9+)	`merged = d1 | d2`
Reverse a list	`xs[::-1]` or `reversed(xs)`
Flatten one level	`[x for row in grid for x in row]`
Sort by key	`sorted(xs, key=lambda p: p.age, reverse=True)`
Deduplicate, keep order	`list(dict.fromkeys(xs))`
Read/write JSON file	`json.load(f)` / `json.dump(obj, f)`
Time a block	`time.perf_counter()` deltas
Default for missing key	`d.get(k, default)`
Numeric literal readability	`1_000_000`

LIST VS TUPLE VS SET VS DICT — WHEN TO REACH FOR WHICH

Need	Choose
Ordered, editable sequence	`list`
Fixed record / dict key / return of multiple values	`tuple`
Fast "is X present?" / remove duplicates	`set`
Look up a value by a key	`dict`

Best Practices & Common Pitfalls

✓ **Do**

Follow PEP 8; auto-format with `ruff format` (or `black`) and lint with `ruff check`. Use descriptive names, small functions, type hints on public APIs, and f-strings for formatting.

✓ **Do**

Use `pathlib` for paths, `with` for resources, comprehensions for transforms, and `logging` (not `print`) in libraries and services.

✗ **Avoid**

Mutable default arguments, bare `except:`, modifying a list while iterating it, and `from module import *`.

⚠ **Watch**

`is` tests identity, `==` tests equality — use `==` for values, `is` only for `None`/singletons. Integer caching makes `a is b` deceptively true for small ints.

PYTHON

```python
# WRONG: shared mutable default persists across calls
def append(item, target=[]):
    target.append(item); return target
append(1); append(2)        # returns [1, 2] -- leaked state!

# RIGHT
def append(item, target=None):
    target = [] if target is None else target
    target.append(item); return target

# WRONG: mutating while iterating skips elements
for x in nums:
    if x % 2 == 0: nums.remove(x)   # buggy
# RIGHT: iterate a copy or build a new list
nums = [x for x in nums if x % 2 != 0]
```

Interview Questions & Answers

What is the GIL and how does it affect concurrency?

The Global Interpreter Lock ensures only one thread runs Python bytecode at a time in CPython. It protects internal state but means threads give no speedup for CPU-bound work — use `multiprocessing` for that. Threads still help I/O-bound work because the GIL is released while waiting on I/O. Python 3.13 introduces an experimental free-threaded build that removes it.

Difference between a list and a tuple?

Lists are mutable and used for homogeneous, variable-length sequences; tuples are immutable, hashable (so usable as dict keys or set members), slightly smaller/faster, and convey "fixed record." A tuple's immutability is shallow — it can still contain mutable objects.

How does Python manage memory?

CPython uses reference counting: each object tracks how many names reference it, and is freed when the count hits zero. A cyclic garbage collector handles reference cycles (e.g. two objects referencing each other). Memory for small objects is pooled by internal allocators.

Explain `*args` and `**kwargs`.

`*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dict. At a call site the same operators unpack a sequence/mapping into arguments. They enable flexible, forwarding-friendly APIs.

What is a decorator?

A callable that takes a function (or class) and returns a replacement, letting you add behavior — timing, caching, auth, logging — without modifying the wrapped code. `@dec` above `def f` is sugar for `f = dec(f)`. Use `functools.wraps` to preserve metadata.

Difference between `is` and `==`?

`==` compares values (calls `__eq__`); `is` compares identity (same object in memory). Use `==` for equality, `is` only for singletons like `None`. Small-int and short-string interning can make `is` accidentally return `True`, so never rely on it for value comparison.

What are generators and why use them?

Functions that `yield` values lazily, producing one item at a time and suspending state between calls. They use constant memory regardless of stream length, so they are ideal for large files, infinite sequences, and streaming pipelines. A generator expression is the lazy analogue of a list comprehension.

Shallow vs deep copy?

> A shallow copy (`list.copy()` , `copy.copy`) duplicates the top-level container but shares the nested objects. A deep copy (`copy.deepcopy`) recursively duplicates everything. Choose deep copy when nested mutable state must be independent — but note it is slower and can fail on unpicklable objects.

> **How do you handle exceptions cleanly?**
>
> Catch the narrowest exception you can handle, prefer EAFP (try/except) over pre-checking, use `finally` or context managers for cleanup, chain with `raise ... from e` to preserve cause, and never use a bare `except:` . Define custom exception classes for your domain.

> **What does `if __name__ == "__main__":` do?**
>
> When a file is run directly, its `__name__` is `"__main__"` ; when imported, it is the module's name. The guard lets a file expose functions for import while also being runnable as a script, without executing the script body on import.

> **Explain list vs generator memory trade-off with an example.**
>
> `[x*x for x in range(10**8)]` allocates ~800MB up front; `(x*x for x in range(10**8))` allocates almost nothing and yields values on demand. Use the list when you need random access/reuse, the generator when you iterate once over large or streaming data.

> **How does method resolution order (MRO) work?**
>
> Python uses the C3 linearization algorithm to order base classes for attribute/method lookup under multiple inheritance. `ClassName.__mro__` shows the order; `super()` follows it, enabling cooperative multiple inheritance where each class's method calls the next in the chain exactly once.

Further Learning

- **Effective Python** (Brett Slatkin) — 90 concrete best-practice items; the single best intermediate book.
- **Fluent Python** (Luciano Ramalho, 2nd ed.) — deep dive into the data model, generators, and concurrency.
- Read the official **docs.python.org** tutorial and the **PEP 8** style guide end to end.
- Practice DSA in Python on LeetCode/NeetCode to cement idioms under time pressure.
- Adopt **ruff** (lint + format), **mypy/pyright** (types), and **pytest** in every project.
- Next languages/tools for an AI engineer: NumPy & pandas internals, then PyTorch, then `asyncio` for serving models.